MPC

Jil Zerndt FS26
Multicore and Parallel Programming

L1 Introduction & Task Decomposition

Vocabulary & Platform Model

Compute Hierarchy Host **delegates to** a Compute Device.
- **Host**: controller that delegates
- **Device**: receives the work
- **CU** → **PE** → **FU** (units inside)
- **UE**: process/thread running a task

"Task" (dual use)
- Linux sense: a process **or** a thread
- parallel sense: a part of a parallel program (a unit of work)

Recipe: Map Platform → Terminology

Find the host
which side delegates work.

Find the device
which side receives it; inside it CU → PE → FU.

Identify UEs
each spawned process/thread; a **task** is one work slice it runs.

Ex. CB1 / Jetson terminology Programming on the CB1, delegating to the Jetson Nano. Map the platform; and what are the two meanings of "task"?
Solution: CB1 = **Host**; Nano = **Device**; Nano SMs = CUs; CUDA cores = PEs; ALU/FPU = FU; spawned threads = UEs. "Task" = a process/thread (Linux) **or** a piece of parallel work (here, one Sobel slice).

General Solutions (Algorithm Structure)

Pattern Hierarchy Worked top→bottom: Finding Concurrency → Algorithm Structure → Supporting Structure → Implementation Mechanisms.

Algorithm-Structure Decision Tree Pick by the **major organising principle** × **structure**:
- Organise by **Task**: Task Parallelism (linear) · Divide & Conquer (recursive)
- Organise by **Data**: Geometric Decomposition (array/linear) · Recursive Data
- Organise by **Flow**: Pipeline · Event-Based Coordination

Recipe: Choose an Algorithm-Structure Pattern

Organising principle
what dominates tasks, data, or data-flow?

Structure
linear/array or recursive?

Map
task+linear → Task Parallelism; task+recursive → Divide & Conquer; data+array → Geometric Decomposition; data+recursive → Recursive Data; flow → Pipeline / Event-Based.

Ex. Pick the pattern (a) Sobel applies the same kernel to every image region. (b) Quicksort recursively splits an array. Which Algorithm-Structure pattern each?
Solution: (a) Organise by **data**, array ⇒ **Geometric Decomposition**. (b) Organise by **task**, recursive ⇒ **Divide & Conquer**.

Top-Down Pattern Path

Layer → Pattern (Sobel)
- Finding Concurrency → Task Decomposition + Dependency Analysis
- Algorithm Structure → Task Parallelism
- Supporting Structure → Fork / Join
- Implementation Mechanisms → Processes & Threads

Recipe: Trace Patterns Top-Down Walk the four layers in order, choosing one pattern per layer for your problem: expose concurrency → organise the algorithm → pick supporting structures → map to threads/processes.

Ex. Trace Sobel through the layers Name the chosen pattern at each of the four layers for Sobel.
Solution: Finding Concurrency: **task decomposition + dependency analysis**; Algorithm Structure: **task parallelism** (and geometric decomposition for the data view); Supporting Structure: **fork/join**; Implementation: **processes & threads**.

Task Decomposition

Functional vs Loop Splitting
- **Functional**: split by named function (few tasks)
- **Loop/data**: split per-element loop by region (scales)

Solution-Space Criteria
- find as many identifiable tasks as possible
- enough to keep all cores busy
- tasks as independent + distinct as possible

Sobel Pipeline (running example) `read_image` → `rgb_2_grey` → {`sobel_x` ∥ `sobel_y`} → `sobel_all` → `display_image` only `sobel_x` & `sobel_y` are independent.

Sobel Operator $G_x = \begin{bmatrix} -1 & 0 & +1 \\ -2 & 0 & +2 \\ -1 & 0 & +1 \end{bmatrix}$, $G_y = \begin{bmatrix} -1 & -2 & -1 \\ 0 & 0 & 0 \\ +1 & +2 & +1 \end{bmatrix}$; $|G| = \sqrt{G_x^2 + G_y^2} \to |G_x| + |G_y|$

Recipe: Task-Decompose an Algorithm

Draw the CFG
nodes = functions, edges = data deps.

Count distinct actions
= functional tasks.

Find independent tasks
no path between ⇒ concurrent.

Try both splittings
functional vs loop/region.

Check solution space
enough tasks for all cores; as independent + distinct as possible.

Ex. Decompose Sobel Split Sobel into tasks. How many distinct actions? Independent tasks? Functional or loop splitting?
Solution: 6 functional tasks; **6** distinct actions; only `sobel_x`,`sobel_y` independent. Functional split = limited (2-way); **loop/region splitting** scales to all cores.

CFG vs Call Graph

CFG vs Call Graph
- **CFG** (control-flow graph): **all possible** execution paths static; used by compilers/optimisers
- **Call graph**: the **actual** execution/calls dynamic; used by profilers

Recipe: Tell CFG from Call Graph Ask: **possible** paths (static, compiler/optimiser) → CFG; **actual** calls observed (dynamic, profiler) → call graph.

Ex. Which graph? (a) A profiler reports which functions actually called which, and how often. (b) A diagram of every branch the program could take. Which is which?
Solution: (a) **Call graph** (actual, dynamic, profiler). (b) **CFG** (all possible paths, static).

Dependency Analysis (Finding Concurrency)

Group / Order / Share
- **Group**: tasks with same data/constraints
- **Order**: producer before consumer
- **Data-sharing**: classify each datum

Data-Sharing Types
- read-only
- effectively-local
- read-write (accumulate / multi-read-single-write)

Ex. Dependency analysis for Sobel Which sub-pattern matters most? Group/order/share for Sobel?
Solution: **Data Sharing**. Group `sobel_x/y` (same grey input); order `sobel_all` after both; grey = read-only, kernel = effectively-local, `Gx/Gy` = read-write, `sobel_all` = **accumulate**.

Task-Parallelism Dependencies

Dependencies

- **Embarrassingly parallel**: no dependencies among tasks
- **Order** dependency: must be enforced (producer→consumer)
- **Data** dependency:
  - **removable** via code/loop transformation
  - **separable** replicate the data structure (replicated data), then combine

Recipe: Analyse Task Dependencies

None?

independent → embarrassingly parallel.

Order?

enforce producer before consumer.

Data?

try to make it **removable** (transform) or **separable** (replicate + combine, e.g. reduction).

Ex. Classify the dependencies Tasks all add into one shared `sum`. What dependency is this, and how do you parallelise it?
Solution: A **data dependency** on `sum`. It is **separable**: give each UE a private partial sum (replicated data), then combine i.e. a **reduction**. (By contrast `sobel_x`‖`sobel_y` are embarrassingly parallel; `sobel_all` has an **order** dependency on both.)

Scheduling (Task → UE)

Schedule Types

- **Static**: assigned a priori; low overhead; needs predictable/equal load
- **Dynamic**: task queue, UE grabs next when free; for unknown load
- **Work-stealing**: idle UE takes from others' queues; adds complexity

Task-Parallelism Checks

- compute > management overhead
- #tasks ≥ #PEs
- minimise dependencies

Recipe: Choose a Schedule equal/predictable load → **static** (low overhead); uneven/unknown → **dynamic** queue; idle UEs + imbalance → **work-stealing** (at extra complexity).

Ex. Pick a schedule 6 equal Sobel chunks on 4 cores. Static or dynamic? Why?
Solution: 6 on 4 leaves an uneven last round, so prefer **dynamic** (or make #chunks a multiple of 4). If chunks are truly equal **and** divide the cores, **static** wins on low overhead. **Work-stealing** helps if some chunks run long.

Supporting Structure & Implementation

Fork–Join

Clone code into concurrent tasks, then **join** (wait). **Not** POSIX `fork()`; suits recursive/irregular work.

Process vs Thread

- **Process**: own address space, MPU-isolated, `fork()` (CoW)
- **Thread**: shares memory, cheap → thread pool

Ex. Fork-join & process vs thread Is fork-join a good fit for Sobel? Process or thread for the workers?
Solution: Fork-join is a **poor fit** for a fixed pipeline (it suits irregular/recursive work). Use **threads**: they share the image memory (cheap comms) and are cheap to create vs processes.

Exam FS23 Choosing a UE

Recipe: Choose a UE (Process vs Thread)

Shared data?

yes → threads (shared memory, cheap comms); isolation needed → processes.

Cost of UE

thread create ≪ process (`fork`/CoW).

Cost of IPC

threads share memory; processes need pipes/shm/messages.

Cost of distributing chunks

threads = pointers; processes = copy/map.

UE → core

pin/affinity to keep caches warm.

Exam FS23 Q3.3 Choice of UE (5P) Sobel on 1024×1024 streamed images, 4-core SMP Linux, at least 6 chunks. Explain your UE choice cost of UE, IPC cost, cost of distributing chunks to UEs, cost of distributing UEs to cores.
Solution: Choose **threads**: cheap to create vs a process (1P); IPC via shared memory is cheap image already in one address space (1P); chunks distributed as pointers/offsets, near-zero cost (1P); pin threads to cores (affinity) for UE→core mapping (1P); structured reasoning throughout (1P).

Exam FS23 **Q1** = "Trivia", 10 single-point questions (the paper leaves the content blank). Expect short recall items from any lecture revise the **definitions** and **concepts** boxes throughout.

L2 Data Decomposition

General Solutions (Algorithm Structure)

Algorithm-Structure Decision Tree Pick by the **major organising principle** × **structure**:
- Organise by **Task**: Task Parallelism (linear) · Divide & Conquer (recursive)
- Organise by **Data**: Geometric Decomposition (array) · Recursive Data
- Organise by **Flow**: Pipeline · Event-Based

Recipe: Choose an Algorithm-Structure Pattern

Organising principle
tasks, data, or data-flow?

Structure
linear/array or recursive?

Map
task+linear → Task Parallelism; task+recursive → Divide & Conquer; data+array → Geometric Decomposition; data+recursive → Recursive Data.

Ex. Pick the general solution A streamed 1024×1024 greyscale image, same filter applied per pixel. Which general solution, and why not task parallelism?
Solution: Organise by **data**, array structure ⇒ **Geometric Decomposition**. Task parallelism gives only a handful of functional tasks; the data view scales with the number of image regions.

The Top-Down Path

Data Decomposition Split the **data structure** (image array) into chunks updated independently. Scales with #chunks.

Ghost / Halo Cells Read-only copy of a neighbour's boundary (the 3×3 kernel reads 1 row/col out). Exchanged a-priori or overlapped with compute.

The Path (and its justification)
1. **Data Decomposition** large array, same op per region
2. **Dependency Analysis** group/order/share; mostly read-only + thin ghost halo
3. **Geometric Decomposition** local-neighbourhood stencil
4. **Loop Parallelism** split the per-pixel loops
5. **Distributed Array** distribute chunks to UEs
6. **SPMD** one program per UE, index by rank

Recipe: Justify the Top-Down Path For each layer state **why** the pattern fits: expose concurrency (data vs task) → analyse dependencies (what is shared) → organise the algorithm (stencil ⇒ geometric) → choose supporting structures (loop parallelism + distributed array) → map to UEs (SPMD).

Ex. Justify the path for Sobel Walk the top-down path and justify each pattern choice.
Solution: **Data Decomposition** (image is a big array, op is per-region); **Dependency Analysis** (grey = read-only, only a 1-px ghost halo shared); **Geometric Decomposition** (local 3×3 stencil); **Loop Parallelism** (split the pixel loops); **Distributed Array** (hand chunks to UEs); **SPMD** (each UE runs the same code, indexed by rank).

Granularity

Granularity Knob Chunk **size** and **number** a tunable to fit the solution onto different hardware.

Surface vs Volume equal volume $N^2/4$: square surface $2N$ vs strip $\frac{5N}{2}$ ⇒ **square wins**. 10×10: square 25 calc / 20 transfers; strip 20 / 25.

Impact of Granularity
- computation scales with **volume** (cells)
- boundary/ghost cost scales with **surface**
- dependency-management time must be ≪ computation

Recipe: Tune Granularity

Enough chunks
≥ cores (more for balancing).

Big enough
so dependency-management overhead ≪ per-chunk computation.

Compact shape
minimise surface/volume (square > strip).

Ex. Estimate the overhead 1024×1024 image in 1024×128 strips. Estimate communication vs computation.
Solution: Compute ≈ $1024 \cdot 128 = 131072$ px (volume); ghost ≈ $2 \times 1024 = 2048$ px (surface) ⇒ comm/compute ≈ **1.6%** overhead negligible, so this granularity is fine.

Efficiency & Mapping Chunks to UEs

Efficiency Maximise useful **work** vs management **overhead**; pick granularity (LZ3) accordingly.

Distributed Array More chunks than UEs, assigned block-cyclic / round-robin so each UE owns several scattered chunks → smoother load.

Recipe: Map Chunks to UEs

Over-decompose
#chunks > #UEs.

Assign
block-cyclic / round-robin (or a dynamic queue).

Locality
pin UEs to cores so caches stay warm.

Ex. Map 6 chunks to 4 UEs How do you map 6 chunks onto 4 UEs for good load balance?
Solution: Over-decompose ($6 > 4$) so a UE that finishes early takes the next chunk; assign round-robin or via a **dynamic queue**; pin UEs for cache affinity. Equal chunks divisible by UEs ⇒ static is cheaper.

Communication of Indices

Index Mapping UEs work on **local** copies/indices, but the array has **global** indices.
- the global↔local mapping must be communicated to each UE
- align computation with **locality** (cache/memory) process spatially-related data together

Recipe: Handle Index Mapping

Offset
give each UE its global base index (local r → global r+offset).

Ghosts
communicate neighbour boundary indices for the halo.

Locality
keep each chunk spatially contiguous to stay cache-friendly.

Ex. Local vs global indices A UE owns rows 128–255 of a 1024-row image. What index info does it need, and why does locality matter?
Solution: Its global **offset** (128): local row r maps to global $r + 128$. It also needs neighbour boundary rows (**127** and **256**) as ghosts. Processing its contiguous band keeps spatially-related data in cache (cache/memory locality).

Exam FS23 Q3: Sobel Data Decomposition

Scenario Sobel on 1024×1024 streamed greyscale; 4-core SMP Linux; at least 6 chunks.

Exam FS23 Q3.1 Design pattern (1P) Explain your choice of design pattern.
Solution: **Data decomposition**: the image is a large array and the same Sobel op applies independently to regions. Use more chunks than cores (**quantum**) for **load balancing**; respect the **sequence of operations** (stream in → filter → out). State assumptions (greyscale, equal chunks).

Exam FS23 Q3.2 Dimension the chunks (4P) Draw the chunks, dimension them in pixels, explain the dimensioning.
Solution: E.g. **8 horizontal strips of 1024×128 px**: ≥ 6 chunks and > 4 cores ⇒ load balancing; 1024 divides evenly; row-major friendly (contiguous, simple ghost rows). (Square 512×256 tiles also valid lower surface/volume.)

Exam FS23 Q3.5 Ghost cells / cost / loop indices (7P) What is a ghost cell? Estimate computation vs communication for your chunk size. What are the loop indices?
Solution: **Ghost cell** = a boundary cell required by two chunks (a 1-row/col halo for the 3×3 kernel). For a 1024×128 strip: compute ≈ $1024 \cdot 128 = 131072$ px × ∼ 4 MACs (for G_x/G_y); ghost ≈ $2 \times 1024 = 2048$ px ⇒ comm/compute ≈ 1.6% (**surface** vs **volume**). Loops run over the interior with a 1-px border: `for y=1..H` `for x=1..W`, reading neighbours $x-1..x+1$, $y-1..y+1$.

L3 OpenMP

Introduction & Model

What OpenMP Is Three elements:
- runtime library (implementation-dependent)
- compiler directives `#pragma` (ignored if unsupported)
- environment variables (change behaviour at run)

Execution Model
- shared-memory; Fork-Join + SPMD
- a directive opens a **team** of threads
- **implicit barrier** at end of every parallel region

Recipe: Build a Parallel Region

Open a team
`#pragma omp parallel { ... }`.

Share work
add `omp for` (else every thread repeats the block).

Mind the barrier
threads join at the implicit barrier (drop it with `nowait`).

Ex. Components & barrier Name the three components of OpenMP. What happens at the end of a parallel region?
Solution: **Runtime library**, **compiler directives**, **environment variables**. At the region's end there is an **implicit barrier** all threads join before the master continues.

Data Scope & Worksharing

parallel vs parallel for
- `parallel` alone: each thread runs the **whole** block (replicate)
- `parallel for`: iterations **split** (worksharing)

Scope Clauses
- `shared`: one copy (race risk)
- `private`: per-thread, uninit, discarded
- `firstprivate/lastprivate`: seed / copy out
- `reduction(op:v)`: private partials, combined

Recipe: Predict OpenMP Behaviour

Read directives
replicate vs worksharing.

Scope each var
shared / private / reduction.

Find races
a shared var written by > 1 iteration → indeterminate.

Count
multiply by thread count only in replicated regions.

Ex. LZ2a (s7) How many prints?

```
1 #pragma omp parallel                  //
      (A)
2 for (int i=0;i<n;i++) printf("Hi\n");
3 #pragma omp parallel
4 #pragma omp for                       //
      (B)
5 for (int i=0;i<n;i++) printf("Hi\n");
```

With T threads and $n = 5$, how many lines each?
Solution: (A) $T \times 5$ (loop replicated on every thread); (B) 5 (iterations shared once). `parallel` makes threads; `for` splits work.

Ex. LZ2b (s8) Value of `b`?

```
1 int a=7,b=0,n=10;
2 #pragma omp parallel for shared(a,b)
3 for(int i=0;i<n;i++) b=a+i;           //
      section 1
4 #pragma omp parallel for shared(a)
      private(b)
5 for(int i=0;i<n;i++) b=a+i;           //
      section 2
```

Solution: S1 shared: **race** → any of 7..16. S2 private: copies discarded → global `b` unchanged. Final `b` = whatever S1 left.

Ex. LZ2c (s9) Reduction Sum `a[0..8]` with no race; how does `reduction` work?
Solution: `#pragma omp parallel for reduction(+:sum)`: each thread builds a **private partial** (e.g. `a[0..2]`, `a[3..5]`, `a[6..8]`); after the join OpenMP adds the partials.

IPC Synchronisation (Critical Sections)

Critical Section Code only one thread may enter at a time (serialised). Entry must be uninterruptable (`lock()`); careless use → deadlock/livelock.

Mechanisms
- ISA **atomic** (test-and-set)
- `omp atomic` (one op) / `omp critical [name]` (block)
- `sem_*` (processes), `pthread_mutex_*` (threads, owner unlocks)
- `omp_*_lock` (expensive)

Recipe: Pick a Sync Primitive single memory op → `atomic`; a block → `critical`; accumulate → `reduction`; processes → semaphore; threads → mutex; reusable handle → omp lock.

Ex. Increment a shared counter Give three ways to make `count++` safe across threads, and the cheapest.
Solution: `#pragma omp atomic`, `#pragma omp critical`, or a mutex/semaphore. **Cheapest = `atomic`** (uses a HW test-and-set; `critical`/locks are heavier).

Thread Synchronisation (Barriers)

Barriers
- implicit at end of a parallel region
- `#pragma omp barrier` = explicit wall
- `nowait` drops an implicit barrier
- `ordered` = run the marked part in loop order

Barrier Stages
- arrival (trapping): wait-queue **or** busy-loop
- release: all trapped threads let go at once

Recipe: Control Barriers need a wall → barrier; want overlap (no wait) → `nowait`; need ordered output → `ordered`; tune spin-vs-block via `OMP_WAIT_POLICY`.

Ex. (s18) Barrier internals On arrival, wait-queue or busy-loop which is preferable? Once released, who goes first?
Solution: **Depends**: short wait + spare cores → busy-loop (spin); long/oversubscribed → wait-queue (block). Release order = **OS scheduler, nondeterministic**; use `ordered` if order matters.

Scheduling & Task Management

Task Management
- `master`: only the master thread runs the block
- `single`: one arbitrary thread runs it (implicit barrier)
- `sections/section`: task parallelism each section is a task

Loop Schedules
- `static[,chunk]`: assigned up-front (even loads)
- `dynamic[,chunk]`: grab from queue (uneven)
- `guided[,chunk]`: shrinking chunks

set via clause / `OMP_SCHEDULE` / `omp_set_schedule`.

Recipe: Choose a Schedule even & predictable → `static` (low overhead); uneven/unknown → `dynamic`; tapering work → `guided`. Use `sections` for distinct concurrent tasks.

Ex. LZ5a (s22) Effect of `nowait` What does `nowait` do on `#pragma omp single`?
Solution: `single` runs the block on one thread with an **implicit barrier**. `nowait` **removes that barrier** → the other threads proceed to "Do some calculations" without waiting (safe only if no dependency).

Ex. LZ5b Which schedule? A 64-iteration loop on 4 threads where iteration cost varies widely. Which schedule?
Solution: `dynamic` (or `guided`) threads grab work as they finish, balancing the uneven load. `static` would leave fast threads idle.

Runtime Interface

Thread-Count Precedence `if` clause → `num_threads` → `omp_set_num_threads()` → `OMP_NUM_THREADS` → default (≈ #CPUs). May exceed cores up to `OMP_THREAD_LIMIT`. Static → exact; dynamic → upper bound.

Recipe: Determine the Thread Count Walk the precedence list top-down; the first one set wins. Query at run with `omp_get_num_threads` / `omp_get_thread_num`.

Ex. Who wins? `OMP_NUM_THREADS=8`, but the region is `#pragma omp parallel num_threads(4)`. How many threads?
Solution: **4** `num_threads` outranks the environment variable in the precedence list.

Exam FS23 Q4: Dot Product (C & OpenMP)

Scenario 220×220 matrix; compute the dot product of every pair of row-vectors → output matrix. Prototype on a reduced 4×4.

Recipe: Sequential Baseline then Parallelise

Baseline

nested loops + an inner MAC; comment; handle edges (bounds, overflow, symmetry).

Parallelise

add `#pragma omp parallel for` (`collapse` nested loops), keep the accumulator **private**, pick a schedule.

Exam FS23 Q4.1 MAC count (1P) How many MAC operations in the reduced (4×4) form?
Solution: $4 \times 4 = 16$ (per the answer key the 4×4 output of dot products).

Exam FS23 Q4.2 Single-core C (6P) Outline commented C for a single-core CPU with standard ALU.
Solution:

```c
// dot product of every row-vector pair, NxN -> NxN (N=4)
for (int i = 0; i < N; i++) {          // first vector (row i)
  for (int j = 0; j < N; j++) {        // second vector (row j)
    long sum = 0;                      // wide type avoids overflow
    for (int k = 0; k < N; k++)        // MAC over elements
      sum += in[i][k] * in[j][k];
    out[i][j] = sum;                   // symmetric: out[i][j]==out[j][i]
  }
}
```

Exam FS23 Q4.3 OpenMP version (3P) Enhance the code for a 4-core machine using OpenMP directives.
Solution:

```c
// split the pair-loops across 4 cores
#pragma omp parallel for collapse(2) schedule(static)
for (int i = 0; i < N; i++)
  for (int j = 0; j < N; j++) {
    long sum = 0;                      // private (declared inside)
    for (int k = 0; k < N; k++)
      sum += in[i][k] * in[j][k];
    out[i][j] = sum;
  }
```

L4 SIMD (Flynn / NEON / MISD)

Flynn's Taxonomy

The Four Classes instruction streams × data streams:
- **SISD**: 1 thread, 1 pipeline, 1 ALU
- **SIMD**: 1 instr, many FUs/lanes
- **MISD**: lockstep / dependability
- **MIMD**: independent cores

SIMD → SIMT → SPMD SIMD is abstracted to **SIMT** (Nvidia, threads in lockstep) and then to the **SPMD** software pattern.

Recipe: Classify with Flynn Count **instruction** streams × **data** streams: 1/1 SISD; 1/many SIMD; many/1 MISD; many/many MIMD.

Ex. Classify these (a) a vector add of 4 lanes; (b) two CPUs running the same code lockstep; (c) a multicore server. Which Flynn class each? What is (a) abstracted to?
Solution: (a) **SIMD** abstracted to SIMT then SPMD; (b) **MISD** (dependability); (c) **MIMD**.

SPMD Pattern

SPMD **Single Program Multiple Data**: one identical program per UE, each with a unique ID/rank used to branch and index its data. It is the software generalisation of SIMD.

Recipe: Write an SPMD Program Init → get unique ID/rank → common code (branch / index by ID) → distribute data → collect + combine → finalise.

Ex. SPMD vs SIMD Why is SPMD called the software cousin of SIMD?
Solution: Both run the **same instructions/program** on **different data**. SIMD does it in hardware lanes (lockstep); SPMD does it across UEs, each picking its data slice by **rank** it relaxes the strict lockstep.

SIMD Vectorisation Motivation

Why Vectorise DSP kernels FIR filter, Fourier transform, dot product, matrix multiply all reduce to **multiply-accumulate (MAC)**. SIMD does one instruction across many lanes in lockstep (e.g. a 4-lane parallel add vs a scalar add).

Recipe: Spot a Vectorisable Kernel Is it a **regular, repeated MAC over arrays** with independent elements? → vectorise. Pointer-chasing / data-dependent control → does not vectorise well.

Ex. Which vectorise? FIR filter, dot product, linked-list traversal which suit SIMD and why?
Solution: **FIR** and **dot product** regular MAC over contiguous arrays. **Linked-list traversal** does not pointer chasing, no regular data parallelism.

NEON Basics (Vector MAC Unit)

NEON Registers V registers (ARMv8):
- **Q** = 128-bit; **D** = 64-bit
- lanes: 2×64 / 4×32 / 8×16 / 16×8
- 32-/64-bit IEEE754; mnemonics in the ARM ISA

Vector MAC Unit
- centres on **multiply-accumulate** (VFMA/VFMS)
- **no divide** only reciprocal estimate (VRSQRTE)
- VFP (scalar FP) now deprecated for vectors

Instruction Format `V{mod} op {shape}{cond}.dt`:
- `mod`: Q (saturating), H (halving), D (doubling), R (rounding)
- `shape`: L (long), W (wide), N (narrow) affects result size
- `.dt`: data type (e.g. `.s16`)

Recipe: Read a NEON Mnemonic Split `V` + `mod` + op + `shape` + `.dt`: V = vector op; `mod`/`shape` tweak behaviour/width; `.dt` sets lane type.

Ex. Decode `VQADD.s16` What does `VQADD.s16` do? And why is there no `VDIV`?
Solution: **Saturating vector add** of signed 16-bit lanes (Q = saturate instead of wrap on overflow). No `VDIV`: NEON has **no divide** use a reciprocal estimate instead.

NEON Intrinsics

Intrinsics & Types Compiler-supported C functions (portable vs raw assembly).
- `uint16x4_t`: 4×16-bit (64-bit)
- `uint16x8_t`: 8×16-bit (128-bit)
- `uint16x4x2_t`: array of two `uint16x4_t`

Lanes The individual vector elements. Type + register width define the op: `vmul_u16` (4 lanes, D) vs `vmulq_u16` (8 lanes, Q).

Recipe: Decode a NEON Intrinsic Pattern `v op q? _ type`: `q` present ⇒ 128-bit Q (else 64-bit D); `type` `<base><bits>x<count>` → **count** = lanes.

Ex. (s19) Decode intrinsics What does `vmulq_f64` do? How many lanes in `uint16x4_t`? What does `vmul_lane_u16(a,b,L)` do?
Solution: `vmulq_f64`: 128-bit lane-wise multiply of **2 doubles** → `float64x2_t`. `uint16x4_t`: **4 lanes**. `vmul_lane`: broadcast `b[L]` to all lanes, multiply by `a`.

Ex. (s20) Lab: vectorise statistics An IoT gateway aggregates a stream (mean, std, skew, kurtosis) to cut comms. Why vectorise, and what is the catch?
Solution: These reduce to repeated MAC over the samples → vectorise with NEON to finish quickly/efficiently.
Catch: they need **divisions** and **floating point**, both expensive on NEON (no divide) compute incrementally and use reciprocal estimates.

MISD / Lockstep

Lockstep Variants
- **tight**: 2 CPUs 0–2 clk apart, slave compares bus, RESET on diff (ABS)
- **2oo3**: one may fail, re-sync on maintenance
- **loose**: async, **dissimilar** implementations
- software pendant: serial or parallel (sync outputs)

Recipe: Recognise a Dependability Architecture Redundant channels running on the **same** data + compare/vote (RESET on mismatch) ⇒ MISD / lockstep. Real example: U96 / Zynq ZU3EG, 2×Cortex-R5F lockstep.

Dependability not Speed MISD spends redundant hardware on **reliability**. Opposite goal to SIMD/MIMD (which buy speed).

Ex. ABS controller An ABS unit runs two CPUs that execute the same code and compare every bus access, issuing a RESET on any difference. Flynn class? Purpose?
Solution: **MISD** (tightly-coupled lockstep). Purpose = **dependability**: detect a fault via the mismatch and fail safe (RESET), not to go faster.

Exam FS23 NEON & Dot Product

Exam FS23 Q2.4 NEON implementation (1P) How is the NEON implemented (RZ/V2L, ARMv8)?
Solution: It's a **v8 architecture** NEON is integrated into the v8 pipeline (not a separate co-processor).

Recipe: Vectorise a Loop with NEON

Load
pack elements into a vector (`vld1q_*`).

Compute
lane-wise op (`vmulq_*`, `vmlaq_*` for MAC).

Reduce
horizontal add across lanes (`vaddvq_*`) for a dot product.

Remainder
handle elements beyond a multiple of the lane count with scalar code.

Exam FS23 Q4.4 NEON dot product (6P) Write commented (pseudo/intrinsic) NEON code for the dot product of each row-vector pair (reduced 4×4).
Solution:

```
// N=4 fits one int32x4 vector exactly
for (int i = 0; i < N; i++)           // first vector
  for (int j = 0; j < N; j++) {       // second vector
    int32x4_t va = vld1q_s32(&in[i][0]);
    int32x4_t vb = vld1q_s32(&in[j][0]);
    int32x4_t vp = vmulq_s32(va, vb); // lane-wise mult.
    out[i][j] = vaddvq_s32(vp);
    // horizontal add of 4 lanes
  }
// edge: if length not a multiple of 4, do the tail scalar
```

Two loops over the pairs; the MAC is one vector op + a horizontal add.

L5 Operating Systems

Process (OS view)
- unit of **resource ownership** (POSIX)
- HW/SW separation via the **MPU**
- scheduler does time-sharing + load-balancing

Recipe: OS Implications of a Process

Ownership
resources isolated → enforced by the MPU.

Scheduling
time-shared + load-balanced by the OS.

Interaction
sharing needs IPC + consistency (synchronisation).

Linux: everything is a task
- processes, threads, kernel threads
- multi-policy scheduler: **RT** or **normal**
- processes interact → share data + ensure consistency

Ex. Why an MPU and a scheduler? What does the OS need to support processes, and what are the two policy classes?
Solution: The **MPU** enforces resource separation (isolation); the **scheduler** time-shares + load-balances. Policies = **RT** and **normal**.

MP Scheduling & Load Balancing

SQMS vs MQMS
- **SQMS**: 1 shared queue cache reload on migration, doesn't scale
- **MQMS**: per-CPU queue prevents lock/cache contention, **cache affinity**, but load-imbalance

Recipe: Balance Multiprocessor Load need cache affinity → MQMS (per-CPU queue); imbalance appears → migrate or **work-steal**; keep tasks on their CPU to preserve warm caches.

Fixing Imbalance
- **migration**: move tasks (partial affinity)
- **work-stealing**: idle queue takes from a busy one

Ex. Imbalanced MQMS CPU0 runs heavy task A while B and D share CPU1; then A finishes. What's the problem and the fix?
Solution: **Load imbalance**: CPU0 goes idle while B/D still cramped on CPU1. Fix with **migration** (move B or D over) or **work-stealing** (idle CPU0 takes from CPU1's queue).

The Three RT Schedulers

RT Schedulers
- `SCHED_FIFO`: high-prio, run-to-completion (prio 1–99)
- `SCHED_RR`: high-prio, time-sliced (prio 1–99)
- `SCHED_DEADLINE`: earliest-deadline-first; can be pinned / bypass balancing

Ex. FIFO or RR? Two equal-priority RT threads must share the CPU fairly. Which scheduler?
Solution: `SCHED_RR` time-slices equal-priority threads. `FIFO` would let the first run to completion before the second starts.

Recipe: Pick an RT Scheduler run-to-completion → `FIFO`; fair time-slice among equal priority → `RR`; explicit per-job deadline → `DEADLINE`.

The Three Scheduler Categories

Categories
- **RT**: `FIFO/RR/DEADLINE`
- **Fair**: `SCHED_OTHER` = EEVDF (←CFS←O(1)←O(n)), `nice`, red-black tree
- **Low**: `BATCH` / `IDLE`

Design Goals
- RT: N highest-prio RT tasks run (N = #CPUs)
- OTHER: resource **utilisation** offload busy → idle CPUs

Recipe: Classify a Scheduler
strict priority → RT; fairness (nice, RB-tree) → `OTHER`/EEVDF; background → `BATCH`/`IDLE`.

Ex. Which category? Which scheduler maintains fairness using a self-balancing red-black tree and the `nice` value? What is its design goal?
Solution: `SCHED_OTHER` (EEVDF/CFS) goal is resource **utilisation** (offloading busy CPUs to idle ones).

Amdahl's Law

Amdahl's Law fraction p parallelisable on N processors:

$$S(N) = \frac{1}{(1-p) + \frac{p}{N}}, \qquad S_{\max} = \frac{1}{1-p}$$

Max may not be reached due to **cache coherence** + **synchronisation** overhead.

Recipe: Apply Amdahl's Law
Find p (parallel fraction) → $S(N) = \frac{1}{(1-p)+p/N}$ → ceiling $S_{\max} = \frac{1}{1-p}$ $(N \to \infty)$.

Ex. Worked $p = 0.9$ on 8 cores: speed-up? maximum? why might you not reach it?
Solution: $S(8) = \frac{1}{0.1+0.9/8} = \frac{1}{0.2125} \approx 4.7\times$; $S_{\max} = \frac{1}{0.1} = 10\times$. Not reached due to **cache-coherence** traffic + **synchronisation** overhead.

Critical Sections & Synchronisation

Critical Section RW-shared data needing protection for consistency. Entry must be **atomic** (one task inside at a time).

Recipe: Protect a Critical Section short wait + spare core → spinlock/ISA atomic; long wait / oversubscribed → OS mutex/semaphore (blocks, frees the CPU).

Atomic-Entry Mechanisms
- disable interrupts/scheduler
- ISA atomic instruction
- software: Peterson / spinlock (busy-wait)
- language: Java `synchronized`
- OS: mutex/semaphore (**blocks**, no spin)

Ex. Protect a shared balance Two processes update a shared bank balance. What is the protected region called? Give a blocking and a spinning way.
Solution: A **critical section**. Blocking: OS **mutex/semaphore** (thread blocks). Spinning: **Peterson/spinlock** or an ISA atomic (busy-wait).

TAS vs TTAS

Test-and-Set Atomic instruction (e.g. `XCHG`) writes 1 and returns the old value in one step; spinlocks loop on it. **Slow**: every attempt locks the bus + invalidates the line.

TTAS spinlock

```
// Test-and-Test-and-Set: read first, then atomic write
do {
  while (locked == true) ;   // spin on a cached READ (cheap)
} while ( TestAndSet(locked) ); // atomic only when it looks free
```

Recipe: Build a Spinlock `TAS` = atomic swap in a loop (bus-heavy). Prefer `TTAS`: spin on a plain read, do the atomic write **only** when the lock looks free → far less coherence traffic, closer to ideal.

Ex. Why TTAS? Why is TTAS faster than TAS under contention?
Solution: TAS spins with atomic **writes** → each attempt locks the bus + invalidates the line. TTAS spins on a cached **read**, writing only when free → **far less bus/coherence traffic**.

Method: Modelling (be aware of) **Petri nets**: places, transitions and tokens for modelling concurrent/synchronised systems [s14].

Exam FS23 Q3.4 Scheduler on the cores (6P) Sobel, 4-core SMP, at least 6 chunks, streamed images. Choose a scheduler; state assumptions.
Solution: **Assumptions**: equal chunk size, CPU-bound, soft-real-time stream. 6 chunks on 4 cores ⇒ uneven last round, so use **dynamic** load balancing (or make #chunks a multiple of 4). For a per-frame deadline use an **RT** policy (`SCHED_FIFO/RR`) with threads **pinned** to cores (affinity); otherwise `SCHED_OTHER` suffices. Coherence: chunk count > cores enables the balancing.

L6 Memory

Cache & Paging

Cache Fast memory near/in the CPU, **transparent** to the programmer.
- hit → fast
- miss → fetch a whole **line**, also cache it
- write-back uses a **dirty bit**

Paging Address space → pages (4K/2M); physical → frames.
- only **used** pages loaded (any frame)
- **resident set** = pages in memory
- **working set** = pages in use now

Recipe: Reason about Cache/Paging cache: hit serve / miss fetch a line (exploit locality). paging: page→frame via page table; keep only the working set resident.

Ex. Why is a[1] fast after a[0]? Reading `a[0]` misses but `a[1..15]` are fast. Why? And resident vs working set?
Solution: The miss on `a[0]` fetches the **whole line** (incl. `a[1..15]`) → subsequent reads are **hits** (spatial locality). Resident = pages in memory; working = pages in use now.

The Memory Access Walk (worked)

Recipe: Trace a Memory Access

Cache
hit → done; miss → continue.

MPU
access allowed?

MMU/TLB
hit → frame; miss → walk page tables.

Page fault
empty → MM loads frame, updates page dir.

Fill
TLB caches mapping; cache reads a line; a write sets the **dirty bit**.

Access Chain CPU pipeline (IF·ID·EX·MEM·WB) → Cache → **MPU** (rights) → **MMU** (**TLB** else page-table walk) → **page fault** → MM software.

Ex. (s6–12) Trace the code Walk `mov ea,var1; mov eb,var2; add ea,eb; mov var3,ea`. Why is the first fetch slow but the next fast?
Solution: First fetch (`0x100`): cache miss → MPU → MMU → TLB miss → walk empty → **page fault** (load frame #2). Fill = whole line (I1–I4) + TLB mapping ⇒ `0x104`,`0x108` are **cache hits**. `var1` (`0x400`): 2nd page fault (frame #8); `var2` = hit; `mov var3` = write → **dirty bit**.

Thrashing

Two Types
- **cache**: many lines map to one slot → evict/miss loop
- **disk**: too few frames → page evict/reload loop

Effect Both stall the CPU waiting on much slower memory/disk.

Recipe: Diagnose Thrashing repeated misses on the **same cache slot** → cache thrash (fix: layout/associativity); constant page evict/reload (too few frames) → disk thrash (fix: fewer resident processes / more memory).

Ex. Same-slot loop A loop alternately touches two variables that map to the same cache line, then needs the first again. What happens?
Solution: **Cache thrashing**: each access evicts the other → repeated misses → CPU stalled.

Cost of Cache-Oblivious Code

Context-Switch Cost
- cache **high** (flush + write-back dirty)
- TLB **high** (flush)
- MPU / page-tables: medium; pipeline: low

Recipe: Estimate Switch Cost cache + TLB dominate (must be flushed/re-warmed). Expect CPI to spike right after a switch, then fall as caches re-fill.

Ex. CPI after a switch After a context switch, why does CPI spike then fall over the next thousands of instructions?
Solution: The cache + TLB are flushed → a burst of **misses + page-walks** → high CPI; as they **re-warm**, CPI drops back. This is the cost of not programming cache-aware.

Multicore L1/L2 (Read & Writeback)

Cache Writeback A dirty line is written back when (1) the controller decides, or (2) it needs the line (eviction). Then dirty→0; eviction sets valid→0, refill, valid→1.

On a Shared L2
- read: line pulled L2→requesting core's L1 (others keep copies)
- **flushing L1 requires flushing L2**; L1→L2 may trigger L2→memory

Recipe: Will a Writeback Happen? Check the dirty bit: dirty + (controller decision or eviction) → write back first. Clean line → just refill. Multicore: an L1 flush cascades to L2.

Ex. Read then flush (a) Two cores read `var1` what's in their caches? (b) When does a dirty line actually get written back?
Solution: (a) Both get a **copy** in their L1 from the shared L2 read-sharing is fine, no invalidation until a write. (b) When the controller decides **or** it needs the line (eviction) and the line is **dirty**.

Coherence & Consistency

Coherence Visibility of a write across cores. A write **will** become visible and writes to **one** location are seen in order. Says nothing about **when**; not in the ISA; necessary not sufficient.

Consistency Ordering of accesses to **different** locations seen by another core.
- sequential / processor / release
- release: order only at sync barriers

Recipe: Coherence vs Consistency One location, "is the write seen and in order?" → **coherence**. Multiple locations, "in what order are they seen?" → **consistency** (strong = all hazards barred, slow; release = order only at barriers, fast).

Ex. The read sequences P1 observes reads 0-1-1-2 while P2 observes 0-2-1-2. What is violated and what is needed? What is release consistency?
Solution: The two cores see writes in **different** order → a **coherence protocol** must make P2 observe P1's write order. **Release consistency** = ordering is enforced **only** at explicit synchronisation (memory barriers), maximising performance elsewhere.

Managing Coherence

Snooping Bus broadcast; every cache controller listens and invalidates/updates matching lines. **Non-scalable**.

Directory Point-to-point. A line has an **owner**; a writer informs the owner, which invalidates sharers, waits for ack, then the write proceeds and the writer becomes owner. **Scalable**.

Recipe: Pick/Trace a Protocol small bus-based system → **snooping** (broadcast + invalidate). many cores → **directory** (owner → invalidate sharers → ack → write → new owner).

Ex. P1 writes a shared line `var1` is cached in both P1 and P2; P1 writes it. Trace it under snooping, then under a directory.
Solution: **Snooping**: P1 broadcasts the write on the bus → P2's copy invalidated. **Directory**: P1 tells the owner → owner invalidates P2 → ack → P1 writes and becomes owner.

False Sharing

False Sharing Two cores write **different** variables that sit in the **same** cache line → the line ping-pongs between caches (useless coherence traffic).

Recipe: Diagnose & Fix False Sharing same line in ≥2 caches + writes to **different** vars → false sharing. Fix: **pad/align** the variables onto separate cache lines.

Ex. Adjacent counters Two cores each increment `v[0]` and `v[1]` (same line). Problem and fix?
Solution: **False sharing**: each write invalidates the other's copy → line ping-pongs. Fix: pad/align so the two vars are on **separate lines**.

ASID / CPU Pinning (worked)

Preserving Cache/TLB State Cache + TLB hold expensive process state. Physical-address tagging or an **ASID** (Address Space Identifier) lets one process's lines survive while another runs.

Recipe: Keep Caches Warm Tag lines by ASID / physical address → lines persist across switches → cheap to re-schedule a process on the same core (**CPU pinning / processor affinity**).

Ex. Cheap re-scheduling How do you make re-scheduling a process on its previous core cheap?
Solution: Preserve its cache/TLB lines via **ASID**/physical tagging and **pin** it to that core (affinity) avoids the flush + re-warm cost.

Memory Architectures

UMA / NUMA / SUMA
- **UMA**: equal access, ∼4-CPU limit, L2/L3 buffer
- **NUMA**: fast local, slow remote via interconnect (ccNUMA)
- **SUMA**: interleaving → best-case UMA

OS Support (Linux)
- HW **cells** → **nodes**; keep work + memory local
- **zonelists** by NUMA distance
- `taskset` / `numactl` / `sched_setaffinity`

Recipe: Classify a Memory Architecture equal access → UMA (bus latency limits to ∼4 CPUs); fast-local/slow-remote → NUMA; interleaved-to-fake-contiguous → SUMA. Keep data local for cache affinity.

Ex. Limits & OS support Why does the traditional bus cap at ∼4 CPUs? Distinguish UMA/NUMA/SUMA. How does Linux handle NUMA?
Solution: Bus latency grows with length → ∼4-CPU sweet spot. **UMA** equal access; **NUMA** fast local + slow remote (ccNUMA); **SUMA** interleaved → best-case UMA. Linux maps **cells**→**nodes**, keeps memory local, uses **zonelists** by distance + `taskset`/`numactl`.

Exam FS23 MMU & Cache Info

Recipe: Explain a HW Unit (define / function / why)

Define

expand the acronym.

Function

state precisely what it does.

Why present/absent

reason from the device type (e.g. a real-time device needs no MMU).

Recipe: Cache Info for Cache-Aware Design To reason about a cache you need: **line size**, **associativity**, total size (and #levels / shared vs private).

Exam FS23 Q2.7 MMU (4P) The CPU has no MMU what does the term mean, what does it do, and why might this device lack one?
Solution: **Memory Management Unit** (1P). Responsible for **virtual memory** address translation, protection, paging (1P). This is probably a **real-time** device, so there is no reason for one (2P).

Exam FS23 Q2.8 Missing cache info (2P) Assessing the platform for CPU-driven edge detection what cache-relevant info is missing?
Solution: **Line size** (1P) and **associativity** (1P).

L7 CPU Architectures

GP-CPU Architectures

Three Architectures
- **Von Neumann**: common I+D memory
- **Harvard**: separate I/D paths
- **Modified Harvard**: Harvard inside the chip, von Neumann outside (IC pins are expensive)

Fundamental Parts
- cache (speeds transfers into the core)
- instruction pipeline + decoder
- ALU

Recipe: Classify a CPU Architecture shared I+D bus → Von Neumann; fully separate → Harvard; **split L1 I/D but unified L2/RAM** → **Modified Harvard**. Datasheet cue: separated I/D caches.

Ex. (s5) A53 architecture A53: L1-I (2-way) and L1-D (4-way) separate, unified L2 (16-way). What architecture?
Solution: **Modified Harvard**: Harvard inside (split L1), von Neumann outside (unified L2/memory).

Parallelising a CPU

Four Ways
- multicore (covered)
- multiple FUs (ALUs/multipliers) → **data** parallelism
- multiple pipelines → **instruction** parallelism
- heterogeneous PEs (DSP, GPU)

Recipe: Identify the Parallelism Mechanism multiple FUs (VLIW) = data; multiple pipelines (superscalar) = instruction; idle slots filled by a 2nd thread = SMT; separate cores = multicore/AMP.

Ex. Name the four List four ways to add parallelism inside a CPU and the kind each provides.
Solution: **Multicore**; **multiple FUs** (data); **multiple pipelines** (instruction); **heterogeneous PEs** (DSP/GPU). The ALU is not the only option (e.g. FPU/MAC for DSP work).

Multiple FUs: VLIW / DSP

DSP Application-specific FUs: MAC (.M), data access (.D), GP-ALU (.L), shifter/ALU (.S). Harvard memory.

Recipe: Recognise VLIW / DSP separate program/data bus keeping a MAC fed → Harvard DSP; one wide instruction the **compiler** packed per FU → VLIW (+ software pipelining).

VLIW Compiler packs serial instructions into one wide word for the FUs.
- needs many registers + bandwidth
- large, binary-**incompatible** code

Software Pipelining Overlap loop iterations (prologue / kernel / epilogue) to cut pipeline stalls; with VLIW gives very tight loop kernels.

Ex. (s8) DSP with .M/.D/.L/.S What memory architecture is this? How does execution work?
Solution: **Harvard** (separate program/data bus). Execution = **VLIW**: the compiler packs one op per FU into a wide word, run in parallel (+ software pipelining). Big, incompatible code.

CISC/RISC & Hazards

CISC vs RISC
- **CISC**: complex instructions, microcoded
- **RISC**: load/store, many simple instructions, pipeline- and compiler-friendly

Hazards
- structural (resource conflict)
- data (dependency)
- control (branch)

Recipe: Spot a Hazard two instructions need the same unit → structural; one needs another's result → data; a branch changes the path → control.

Ex. Why RISC pipelines better Why is RISC more pipeline-friendly than CISC? Name the three hazard types.
Solution: Uniform load/store instructions fit a fixed pipeline cleanly (CISC needs microcode for variable, complex instructions). Hazards: **structural / data / control**.

Superpipeline vs Superscalar

Superpipelining More / finer pipeline stages → instructions issued faster.

Superscalar More pipelines → several instructions per cycle (scheduling becomes the issue).

Recipe: Tell Them Apart deeper (finer stages, faster issue) = superpipeline; wider (more pipelines, several/cycle) = superscalar.

Ex. Distinguish What is the difference between superpipelining and superscalar?
Solution: **Superpipeline** = more/finer stages (issue faster); **superscalar** = more pipelines (several issued per cycle).

In/Out-of-Order Execution

Three Modes
- in-order issue, in-order execution
- in-order issue, out-of-order execution
- out-of-order issue, out-of-order execution

Recipe: Identify the Mode Ask whether **issue** and **execution** keep program order. Both ordered = simplest; OoO issue + OoO exec = most aggressive (best unit utilisation, most hardware).

Ex. Which is most aggressive? Name the three superscalar scheduling modes; which extracts the most parallelism?
Solution: In-order/in-order; in-order issue/OoO exec; **OoO issue + OoO exec** (the most aggressive, needs the most scheduling hardware).

Co-processors

Operation CPU runs recognised instructions; passes others to the co-pro.
- unrecognised → exception
- own clock + memory channels (NEON) → parallel execution

Interface & Sync
- own ISA; instructions via pipeline, data via registers
- 8087 + 8086: the 8086 **stalls** on `FMUL`
- future uncertain: costly decoders + compiler upkeep

Recipe: Reason about a Co-processor CPU offloads unknown instructions; co-pro has its own ISA/clock/memory; the two need synchronisation (CPU may stall); good only for generic ops.

Ex. 8087 / 8086 How does the CPU handle an instruction it doesn't recognise, and what's the catch with the 8087?
Solution: It **passes it to the co-pro**; if unrecognised there too → exception. Catch: the 8086 **stalls** while the 8087 performs `FMUL` (a pipeline/sync effect).

SMT (Hyperthreading)

SMT A core runs 2 threads to use otherwise-idle resources (4 cores → 8 threads).
- **needs a superscalar** core (spare issue slots)
- ∼30% speedup (cache-stall dependent); OS sees 2 cores; security issues

Recipe: Reason about SMT SMT fills idle issue slots with a 2nd thread ⇒ only worthwhile on a superscalar core; pin a process to a core to control it.

Ex. (s19) Why superscalar? Why does SMT require a superscalar core? Typical speedup?
Solution: Only a superscalar core has **multiple issue slots/pipelines** per cycle to give the 2nd thread; a scalar core has nothing spare. Speedup ∼**30%** (more about cache stalls than FU availability).

AMP Scenarios

Scenarios
- different ISA (PRU/GPU/DSP)
- same ISA (task-driven, master-slave or peer)
- same ISA, different clocks / 64-32 (big.LITTLE)
- different or no OS

Questions to Answer How do the cores boot? Where is their code? How tightly coupled (sync, data, comms)?

Recipe: Set Up AMP same ISA → SMP/big.LITTLE under one OS; **different ISA → separate OS/bare-metal per core + openAMP**. Answer the boot/code/coupling questions first.

Ex. (s20) Different ISA under one OS? Can different-ISA cores run under a single OS? Give two AMP scenarios.
Solution: **No** one kernel is built for one ISA (big.LITTLE works only because it's the **same** ISA). Scenarios: different ISA (specialised PRU/GPU/DSP); same ISA split by task (comms vs I/O); big.LITTLE; different/no OS.

openAMP

remoteproc Boots a remote core: `rproc_boot()` decodes the ELF, places segments in memory, enables resource tables. (Zynq: APU boots Linux via GRUB; RPU R5s start halted.)

rpmsg / virtio IPC: `rpmsg` channels (source/dest, `rpmsg_send` + rx callback) over `virtio` ring buffers (paravirtualisation split driver).

Recipe: Couple AMP Cores with openAMP

Boot
`remoteproc` loads + starts the remote ELF.

Communicate
`rpmsg` messages over `virtio` ring buffers.

Two IPC methods
shared memory + signalling good for asynchronous cores.

Ex. Bring up an R5 on the Zynq On the Zynq UltraScale+, how does the APU start an R5 and talk to it?
Solution: **remoteproc**: `rproc_boot` decodes the R5's ELF and boots it (the RPU starts halted). Communication via **rpmsg over virtio** ring buffers. Zynq = 4×A53 SMP + 2×R5 lockstep + FPGA accelerators.

Exam FS23 Q2: Architectures (RZ/V2L)

Scenario (appendix) RZ/V2L: Cortex-A55 (1.2 GHz) + Cortex-M33 (200 MHz). L1 I/D = 32 KB each (split), L2 = 0 KB, L3 = 256 KB shared; ARMv8.2-A; NEON/FPU; no MMU mentioned; openAMP links A55↔M33.

Recipe: Interpret a Cache Hierarchy

Datasheet
separate I/D caches ⇒ (modified) Harvard.

Per level
L1 closest (often split I/D); L2 unified (may be 0); L3 shared.

Missing for design
line size, associativity, total size.

Exam FS23 Q2.1 Architecture (2P) What architecture does this processor represent and why?
Solution: **Modified Harvard** (1P). Because the data and instruction caches are **separated**, which represents a (modified) Harvard architecture (1P).

Exam FS23 Q2.2 Advantage (2P) What is the advantage of the chosen architecture?
Solution: Instruction and data addressing are separated, so fetching one doesn't hinder the other (1P). Different, more fitting cache architectures can be used (1P).

Exam FS23 Q2.3 L1/L2/L3 (max 4P) What do L1, L2 and L3 represent?
Solution: Cache architecture (1P). L1 split into data + instruction cache (1P). L2 not existent / 0 KB (1P). L3 common (shared) cache (1P).

Exam FS23 Q2.5 M33 to A55 (1P) How can the Cortex-M33 communicate with the A55 cores?
Solution: Via **shared external memory** (1P).

Exam FS23 Q2.6 openAMP IPC (3P) openAMP is in essence two IPC methods which, and why good for M33↔A55?
Solution: **Shared memory** (1P) and **signalling** (1P). Because the two cores are **asynchronous** and openAMP suits that asynchronicity (1P).

L8 GPU Architectures / CUDA / OpenCL

GPU vs CPU

Two Philosophies
- **CPU**: few fat cores, **latency**-optimised
- **GPU**: many simple cores, **throughput**-optimised
- both run "threads of execution"

Why a GPU Wins Many identical independent operations → huge thread count hides memory latency → high throughput.

Recipe: CPU or GPU? massive regular data parallelism, latency-tolerant → **GPU**; branchy, sequential, latency-sensitive → CPU.

Ex. When is a GPU better? When does a GPU beat a CPU, and why?
Solution: When there are many **independent identical** operations: the GPU runs thousands of threads and **hides latency** by switching between them (throughput). A CPU wins on branchy/sequential, latency-sensitive code.

Thread ↔ Loop / Kernel

CUDA Kernel
- CUDA = Nvidia-only C++ extension; "STMD"
- kernel = prologue / kernel / epilogue
- each loop iteration → its own **thread**

Recipe: Convert a Loop to a Kernel

Drop the loop
one thread per iteration.

Index
`i = blockIdx*blockDim + threadIdx` (CUDA) / `get_global_id(0)` (OpenCL).

Guard
`if (i < N)` the grid may overshoot.

OpenCL Vector Add

```
__kernel void vadd(__global float* a,
                   __global float* b,
                   __global float* c) {
    int i = get_global_id(0);
    c[i] = a[i] + b[i]; // one
         work-item per element
}
```

Ex. (s27/28) Loop to kernel Rewrite `for(i<4096) c[i]=a[i]+b[i];` as a kernel; what replaces the loop?
Solution: See the code box: `i=get_global_id(0); c[i]=a[i]+b[i];`. The **index space** (NDRange of 4096) replaces the loop one work-item per element.

Block & Grid Organisation

Threads → Blocks → Grid
- threads grouped into **blocks**; blocks into a **grid**
- must set #threads/block and #blocks
- launched via `«grid,block»`

Why Blocks A block maps a team of threads onto the GPU **HW-independently**; #blocks usually > #processors.

Recipe: Size a CUDA / OpenCL Launch

One thread per element
#threads = #elements.

Block/grid
`threadsPerBlock` a multiple of 32; grid = ⌈elements/block⌉.

Index ranges
`[R][C]` → `threadIdx.x`$\in 0..R-1$, `.y`$\in 0..C-1$.

Edge guard
`if (i<N && j<N)`.

Ex. (s6) Threads for [10][5] Into how many threads does CUDA divide a `[10][5]` matrix op? Range of the threadID?
Solution: **50** threads (10×5). `threadIdx.x`$\in$**0..9**, `threadIdx.y`$\in$**0..4** IDs $(0,0)$ to $(9,4)$.

Execution Model (Grid → SM → Warps)

SM & Warps
- grid → GPU; block → **SM** (Streaming Multiprocessor)
- warp scheduler runs **warps of 32** threads

Fermi SM 32 CUDA cores + 4 SFU + 16 L/S units (GF100 = 16 such cores).

Recipe: Map a Grid onto HW grid → GPU; blocks → SMs (extra blocks **queued**); within an SM threads run as **32-thread warps** scheduled to hide latency.

Ex. What is an SM? What is an SM, how many threads in a warp, and what's in a Fermi SM?
Solution: **Streaming Multiprocessor**; a warp = **32** threads; Fermi SM = 32 CUDA cores + 4 SFU + 16 load/store units.

Limitations & Best Practices

Limits
- available registers
- max active blocks / threads
- shared memory
- warp-scheduler dispatch

Best Practices
- threads are **cheap**
- equal register use → pre-estimate executability
- non-active blocks are **queued**

Recipe: Respect GPU Limits threadsPerBlock a multiple of 32; keep registers/shared-mem per block within the SM budget; over-subscribe with extra blocks (they queue and run as SMs free).

Ex. (s10/11) Multiple of 32? Why make threads-per-block a multiple of 32? What happens to blocks that don't fit on an SM?
Solution: Warps are **32** wide → a non-multiple wastes lanes. Blocks that don't fit are **queued** and run when an SM frees up (threads are cheap, so over-subscribe).

Rendering Pipeline & GPU Evolution

Fixed-Function → Generic GPUs grew out of the **fixed-function shading pipeline** (vertex → raster → fragment). Unified programmable cores made them **generic** (GPGPU); FUs are still developed for graphics. "GPUs echo the shading pipeline."

Recipe: Place a GPU on the Axis dedicated per-stage shader hardware → **fixed-function**; unified programmable cores running arbitrary kernels → **generic** (GPGPU).

Ex. Why generic? What did GPUs evolve from, and why did they become generic?
Solution: From a **fixed-function shading pipeline**. Unifying the shader stages into **programmable cores** let the same hardware run arbitrary compute kernels (GPGPU), not just graphics.

OpenCL Models & Platform

Four Models Platform (HW) · Execution (kernels/NDRange) · Memory · Programming (data/task).

Platform / Driver Host → Compute Device → **CU** → **PE**. The **ICD** (installable client driver) loads the right vendor driver.

Recipe: Recall the OpenCL Models Platform = the hardware map; Execution = how kernels run (NDRange); Memory = the 4 regions; Programming = data vs task parallel. ICD = multi-vendor driver loader.

Ex. Models & hierarchy Name the four OpenCL models and the platform hierarchy.
Solution: **Platform / Execution / Memory / Programming**. Hierarchy: **Host → Device → CU → PE**; the ICD selects the vendor driver.

Work-item / Work-group / NDRange

The Three Terms
- **work-item**: one kernel instance independent + parallel
- **work-group**: scheduling unit on a CU; shares memory; must share a CU
- **NDRange**: the index space; kernel runs once per work-item

IDs / CUDA map
- `global_id` (in NDRange) + `local_id` (in group)
- work-item↔thread, work-group↔block, NDRange↔grid

Recipe: Lay out an NDRange NDRange = total work-items (one per data element); split into work-groups (share local memory, run on one CU); index with `get_global_id` / `get_local_id`.

Ex. (s26) 4096 work-items NDRange of 4096 work-items in groups of 512 relate work-item / work-group / NDRange.
Solution: **NDRange** = the 4096-index space; it splits into **work-groups** of 512 (each scheduled on one CU, sharing local memory); each **work-item** is one kernel instance with a `global_id` (and a `local_id` within its group).

Command Queue

command_queue Kernels are enqueued to the device. **In-order issuance** by the host; **in/out-of-order completion** by the device. The host can insist on in-order completion; otherwise the driver may re-schedule.

Recipe: Reason about the Queue Host controls **issue** order; device controls **completion** order (may reorder for efficiency) unless the host forces in-order.

Ex. (s28) Who controls ordering? Who decides the order of issuance, and who the order of completion?
Solution: The **host** issues in order; the **device/driver** decides completion order (in or out-of-order) unless the host insists on in-order completion.

OpenCL Memory Model

Four Regions
- **Global**: all work-items
- **Constant**: read-only broadcast
- **Local**: per work-group
- **Private**: per work-item

Consistency & Transfer
- **weak** consistency only at sync points (work-group + host)
- host-driven transfer: `clCreateBuffer` + r/w flags

Recipe: Pick a Memory Region shared by all → Global; read-only broadcast → Constant; shared in a work-group → Local; scratch per work-item → Private. The host stages data with `clCreateBuffer`.

Ex. Where + how consistent? Where does a value visible to all work-items live, and what does weak consistency mean here?
Solution: **Global** memory. Weak consistency = updates are only guaranteed visible at **sync points** (a work-group barrier, or host-level for global) not continuously.

OpenCL Synchronisation

Within a Work-group Only No sync **between** work-groups only within one.
- barriers, memory fences
- atomics: `atomic_add/_load/_exchange/_fence`
- mutex built from an atomic

Recipe: Synchronise a Kernel inside a work-group → barrier / fence / atomics. Across work-groups → **not possible** in-kernel: split into separate kernels or synchronise at the host.

Ex. (s33) Coordinate work-items Two work-items in the same group need to coordinate; what about two in different groups?
Solution: Same group: **barrier / memory fence / atomics**. Different groups: **no in-kernel sync** split the work into separate kernels or synchronise at the host.

AMP Recap

Recipe: Set Up AMP (recap) same ISA → SMP/big.LITTLE under one OS; **different ISA → separate OS + openAMP** (`remoteproc` boots the ELF; `rpmsg/virtio` carry messages). Two IPC methods: shared memory + signalling.

Ex. (s40/41) Different ISA under one OS? Can different-ISA cores run under a single OS? How are they coupled?
Solution: **No** one kernel targets one ISA. Couple them with **openAMP**: `remoteproc` + `rpmsg/virtio` (same as L7).

Exam FS23 Q4.5: GPU Kernel

Exam FS23 Q4.5 GPU kernel (6P) Outline a commented GPU kernel for the dot product of every row-vector pair.
Solution:

```
// one thread per output cell (i,j)
__global__ void dot(const int* in, int* out, int N){
  int j = blockIdx.x*blockDim.x + threadIdx.x; // column
  int i = blockIdx.y*blockDim.y + threadIdx.y; // row
  if (i < N && j < N) {                // edge guard
    long sum = 0;
    for (int k = 0; k < N; k++)        // MAC loop
      sum += in[i*N + k] * in[j*N + k];
    out[i*N + j] = sum;
  }
}
// launch: dim3 block(16,16);
//         grid((N+15)/16, (N+15)/16);
```

The grid replaces the two outer loops; the bounds guard handles the edge case.

L9 MPI (Message Passing Interface)

Distributed Architectures

Protocols (rising complexity) single var/record → memory-range → scatter-gather → RPC → distributed objects.

Ex. Struct across machines You send a C struct from a big-endian to a little-endian node. What must you handle?
Solution: **Endianness** conversion, a defined **wire format** (the interface), and channel **reliability**. (MPI derived datatypes help with the format.)

Common Problems
- app ↔ channel interface
- **endianness** of the ends + channel
- channel reliability; app complexity

Recipe: Spot the Distributed-Comms Issues match the protocol to the data shape; **always** handle endianness, a defined wire format, and channel reliability.

The Six Fundamental Operations

The Six
- `MPI_Init` start environment
- `MPI_Comm_size` #processes
- `MPI_Comm_rank` this rank
- `MPI_Send` / `MPI_Recv`
- `MPI_Finalize`

Bindings C / C++ / Fortran. Build with `mpicc`, run with `mpirun`.

Recipe: Skeleton of an MPI Program `Init` → `Comm_size`/`Comm_rank` → branch by rank → `Send`/`Recv` or collectives → `Finalize`.
- distribute data: `Bcast` (all need it) or `Scatter` (split)
- each rank computes its slice; collect with `Gather` / combine with `Reduce`

Ex. Name the six List the six fundamental MPI operations and what each does.
Solution: **Init** (start), **Comm_size** (#procs), **Comm_rank** (my rank), **Send**, **Recv**, **Finalize** (exit).

Communication Models

Communicator & Rank
- communicator = group of processes (default `MPI_COMM_WORLD`)
- rank = a process's number in it

Two Models
- point-to-point (Send/Recv)
- collective (Bcast/Scatter/Gather/Reduce)

Recipe: P2P or Collective? one rank to one rank → point-to-point; one-to-many / many-to-one / all-to-all → a collective (more efficient, e.g. tree Bcast).

Ex. Communicator / rank What is a communicator and a rank? When do you use a collective over point-to-point?
Solution: A **communicator** groups processes (`MPI_COMM_WORLD`); a **rank** is a process's index. Use a **collective** for group-wide data movement (broadcast/scatter/gather/reduce); point-to-point for pairwise messages.

The Run-time

Compile / Run
- `mpicc` (wraps gcc)
- `mpirun -n X` = X slots (SPMD)
- MIMD: `-n 2 a : -n 2 b`
- clusters via ssh + host file

Binding & Scheduling
- bind: `-cpu-set` (+ memory for RDMA)
- scheduling = distributing **pinned** tasks; the OS does the rest

Recipe: Launch an MPI Job build with `mpicc`; run `mpirun -n X` for X ranks; over-subscribing (procs > cores) degrades IPC; pin with `-cpu-set`.

Ex. Build / run / oversubscribe How do you build and run an MPI program on 4 cores? What does oversubscribing do?
Solution: `mpicc -o prog prog.c`; `mpirun -n 4 ./prog`. **Oversubscribing** (more processes than cores) **degrades IPC** tell `mpirun` the slot count.

Blocking vs Non-blocking

Blocking vs Non-blocking
- blocking send: waits until data leaves the output buffer
- blocking recv: waits until all data received
- non-blocking: returns immediately; complete later (Wait/Test)

Recipe: Blocking or Non-blocking? simple / need the buffer reusable now → blocking. Want to **overlap** comms with compute → non-blocking (post `Isend`/`Irecv`, compute, then `Wait`).

Ex. Why non-blocking? When would you choose non-blocking communication?
Solution: To **overlap communication with computation**: post `Isend`/`Irecv`, do useful work, then `Wait` hiding transfer latency.

The Four Blocking Send Modes

Standard & Synchronous
- **Standard** (`Send`): MPI decides buffering; beware long sends
- **Synchronous** (`Ssend`): handshake **safest**, highest sync

Buffered & Ready
- **Buffered** (`Bsend`): app-managed buffer; decouples sender
- **Ready** (`Rsend`): **lowest overhead**; recv must precede send

Recipe: Pick a Send Mode default → Standard; guaranteed/safe → Ssend; decouple sender from receiver → Bsend; receiver already waiting → Rsend.

Ex. Safest vs cheapest Which blocking send mode is safest, and which has the lowest overhead (and its precondition)?
Solution: **Synchronous** (`Ssend`) is safest (handshake). **Ready** (`Rsend`) has the lowest overhead but the **receive must already be posted**.

Non-blocking Send/Recv

Isend / Irecv Return a **request** immediately.
- `MPI_Wait` block until the request completes
- `MPI_Test` poll whether it completed

Recipe: Use Non-blocking `Isend`/`Irecv` → do independent work → `Wait` (or poll with `Test`) **before** reusing the buffer.

Ex. Did it finish? After an `MPI_Isend`, how do you know the transfer is done?
Solution: Check the request: `MPI_Wait` (blocks until complete) or `MPI_Test` (polls). Don't reuse the buffer before it completes.

Message Structure & Data Types

Envelope + Body
- **envelope**: dst/src, tag, comm
- **body**: buffer, count, datatype
- standard types: `MPI_INT/SHORT/LONG...`

Derived Types Custom types via `MPI_Type_create_struct` (e.g. send a C `struct` as one message).

Recipe: Build a Message body = (buf, count, datatype); envelope = (dst/src, tag, comm). Sending a struct? define a **derived datatype** so both ends interpret the bytes identically.

Ex. Two parts + a struct What two parts make up an MPI message, and how do you send a C struct?
Solution: **Envelope** (dst/src, tag, comm) + **body** (buf, count, datatype). For a struct, build a **derived datatype** with `MPI_Type_create_struct`.

Collectives

Bcast Root sends the same data to all (tree-implemented for efficiency).

Scatter / Gather
- **Scatter**: root sends chunks out to nodes
- **Gather**: collect the chunks back

Recipe: Choose a Collective same data to everyone → `Bcast`; split data across ranks → `Scatter`; collect results → `Gather`; combine → `Reduce`.

Ex. Distribute then collect Give every rank the full matrix, then collect each rank's row-results. Which collectives?
Solution: **Bcast** the matrix (all need it) or `Scatter` if each rank only needs its rows then **Gather** the results to the root.

Reduction

MPI_Reduce `MPI_Reduce(send, recv, count, type, op, root, comm)` combines a value from every rank with an op (`SUM/MAX/MIN/PROD...`), element-wise on arrays. `Allreduce` → result to all.

Recipe: Reduce across Ranks each rank holds a partial → pick the op → `MPI_Reduce` to a root (or `Allreduce` to every rank).

Ex. Combine partial sums Every rank computed a partial sum; produce the total at rank 0.
Solution: `MPI_Reduce(&partial, &total, 1, MPI_INT, MPI_SUM, 0, MPI_COMM_WORLD)`. Use `Allreduce` if every rank needs the total.

Cross-lecture threads: Reduction OpenMP(L3)↔accumulate(L1-2)↔`MPI_Reduce`(L9) · SPMD pattern(L2)←SIMD/SIMT(L4)→MPI(L9)→CUDA/OpenCL(L8) · Scheduling L1→L3→L5→L8 · Affinity MQMS(L5)↔ASID(L6)↔`-cpu-set`(L9) · Amdahl(L5) caps all.

Exam FS23 Q4.6 MPI directives (2P) Farm the 220×220 calculation to 200 four-core machines write the MPI directives for the CPU code.
Solution:

```
MPI_Init(&argc, &argv);
MPI_Comm_size(MPI_COMM_WORLD, &size);
MPI_Comm_rank(MPI_COMM_WORLD, &rank);
MPI_Bcast(in, N*N, MPI_INT, 0, MPI_COMM_WORLD);  // all need matrix
// each rank does row band [rank*N/size .. (rank+1)*N/size)
for (int i = rank*N/size; i < (rank+1)*N/size; i++)
  for (int j = 0; j < N; j++) { /* dot -> local_out */ }
MPI_Gather(local_out, rows*N, MPI_INT,
           out, rows*N, MPI_INT, 0, MPI_COMM_WORLD);
MPI_Finalize();
```